

Simulating Memory: Devices to Systems

ECE2.414 Advanced Memory Circuits and Systems, Assignment 1

Hrishikesh Gawas

September 12, 2026

Question. *Should the 2 MB L2 cache in an accelerator be SRAM or STT-MRAM?* The question is carried through five tools, each working at a different level of abstraction: ngspice for the bitcell, CACTI for the SRAM array, NVSim for the STT-MRAM array, Ramulator 2.0 for the DRAM channel behind it, and gem5 for the effect on a running program. Each part's output becomes the next part's input. Stated accuracies are $\pm 5\%$ for ngspice, $\pm 10\%$ for CACTI and Ramulator, and $\pm 20\%$ for gem5, and these compound multiplicatively, so results are reported as ratios between configurations wherever possible.

Sources: <https://github.com/HrishikeshGawasIIITH/SRAM-vs-STT-MRAM>

1 Part A: ngspice, 6T SRAM read margin

1.1 Setup

A single 6T cell on the 45 nm PTM BSIM4 high-performance card. Pull-ups are $W = 0.15 \mu\text{m}$, pull-downs $W = 0.20 \mu\text{m}$ and access transistors $W_A = 0.16 \mu\text{m}$, all at $L = 0.045 \mu\text{m}$. Stored state is $Q = 0$, $\bar{Q} = 1$, set by `.ic v(q)=0` during the operating-point solve. Only Q is forced, because $v(Q) = 0$ does not depend on the supply and so stays valid at every point of the Task 3 sweep; the cross-coupled pair brings $\bar{Q}$ up to whatever rail is applied.

Bitlines are precharged through two PMOS devices of $W = 1.2 \mu\text{m}$ from V_{DD} to BL and BLB, with a $W = 0.6 \mu\text{m}$ equalising PMOS across them. Precharge is active low and releases at $t = 0.75 \text{ ns}$, so both lines are floating at the rail before the read begins. The wordline rises at $t = 1.0 \text{ ns}$ and ΔV is read at $t = 2.0 \text{ ns}$, a 1 ns development window. Wordline and precharge amplitudes are unit pulses scaled by $v(V_{DD})$ so they follow the supply. Transient step is 5 ps over 4 ns.

Bitline load is a parameter, for reasons that Task 1 makes clear. The netlist as given uses $C_{BL} = 180 \text{ fF}$. Average read current is inferred from the measured swing as $\bar{I}_{\text{read}} = C_{BL} \Delta V / \Delta t$.

Table 1: Part A summary at the four measured operating points, $C_{BL} = 180 \text{ fF}$.

Configuration	ΔV @2 ns (mV)	$\bar{I}_{\text{read}}$ (μA)	$\max v(Q)$ (mV)	as % V_{DD}	$v(Q)$ @4 ns (mV)
Task 1 — nominal, $W_A=0.16 \mu\text{m}$, 1.1 V, 27 °C	727	131	206	19 %	2.9
Task 2 — wide access, $W_A=0.24 \mu\text{m}$, 1.1 V, 27 °C	851	153	272	25 %	0.8
Task 4 — nominal, 1.1 V, 85 °C	579	104	221	20 %	15.2
Task 4 — nominal, 0.7 V, 85 °C	192	35	104	15 %	60.0

1.2 Task 1: ΔV at 2 ns, and what the 69 mV reference is

At 1.1 V and 27 °C the measured differential is $\Delta V(\text{BLB}, \text{BL}) = 727 \text{ mV}$. The Lecture 1 reference is 69 mV, so the ratio is 10.5, but the two numbers are not measurements of the same thing.

69 mV is a DRAM sense margin. A DRAM read is charge sharing: a storage capacitor C_s precharged to V_{DD} or ground is connected to a bitline sitting at $V_{DD}/2$, and the resulting signal is

$$\Delta V_{\text{DRAM}} = \frac{V_{DD}}{2} \cdot \frac{C_s}{C_s + C_{BL}}.$$

With $C_{BL} = 180$ fF at 1.1 V, 69 mV corresponds to $C_s = 25.8$ fF, which is an ordinary DRAM storage capacitor. So the reference figure is consistent, it just describes a different mechanism. There is a hard ceiling on it, because the cell can never give the bitline more than its own stored charge, and that is why DRAM needs a sense amplifier with a small offset.

An SRAM read is not charge sharing. The cell is an active driver: the access transistor and the pull-down NMOS form a path to ground and pull the bitline down for as long as the wordline is high. There is no charge-ratio ceiling, only a current and a time. Fig. 1 shows the consequence: BL does not settle at a small differential, it falls to 0.37 V by the sense point and 6 mV by 4 ns. At 2 ns we are measuring an almost complete discharge.

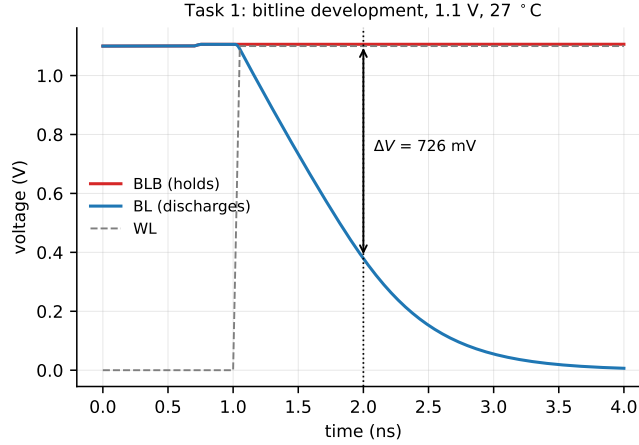


Figure 1: Bitline development at 1.1 V, 27 °C, $C_{BL} = 180$ fF.

The swing corresponds to 131 μ A through a 0.16 μ m device, roughly 0.8 mA/ μ m, which is a normal on-current for a 45 nm high-performance NMOS at 1.1 V. Running the same netlist on the low-power card gives 248 mV instead, 2.9 $\times$ smaller (Table 2), so the device family matters more than anything else in the cell.

Table 2: Effect of the device model card, all else identical.

PTM model card	ΔV @2 ns (mV)	$\bar{I}_{\text{read}}$ (μ A)	ΔV / 69 mV
45 nm HP (high performance)	727	131	10.5 $\times$
45 nm LP (low power)	248	45	3.6 $\times$

The bitline load is the assumption that matters. $C_{BL} = 180$ fF is small for a 2 MB L2. Part B’s winning organisation has a subarray 336 μ m tall, and at CACTI’s SRAM cell height of 0.657 μ m that is 512 cells on each bitline. The wire alone contributes 0.2654 fF/ μ m $\times$ 336 μ m = 89 fF, and the drain junctions of 512 unselected cells add to it, which puts a real column nearer 345 fF. Since ΔV is Q/C_{BL} for a fixed cell current and window, this scales the whole result, so the read was re-measured across a range of column heights (Table 3).

Table 3: Same cell and same timing, varying the column height.

Bitline load	ΔV at 2 ns (mV)				margin at worst corner
	1.1 V/27 °C	1.1 V/85 °C	0.7 V/27 °C	0.7 V/85 °C	
180 fF (as given, $\approx$ 32 cells)	727	579	256	192	7.7 $\times$
260 fF	525	413	180	135	5.4 $\times$
345 fF (CACTI column, 512 cells)	402	315	137	103	4.1 $\times$
700 fF	203	157	70	58	2.3 $\times$
1.8 pF	80	64	35	34	1.4 $\times$

At the CACTI-consistent 345 fF the margin at the worst corner drops from 192 mV to 103 mV, which is 4.1 $\times$ the 25 mV offset rather than 7.7 $\times$. The read still passes, but with roughly half the headroom

the netlist suggests. Only at 1.8 pF, which would need about 3.3 fF per cell and is too high for 45 nm, does the margin fall to $1.4\times$ and start to look genuinely marginal.

1.3 Task 2: does Q move?

It does. At the nominal width Q is lifted to 206 mV, which is 19 % of V_{DD} , because the precharged bitline injects current into the “0” node faster than the pull-down can remove it. At $W_A = 0.24 \mu\text{m}$ the disturb grows 32 % while ΔV improves only 17 %. The access transistor sets the read current and the disturb at the same time, so widening it helps one and hurts the other.

The cell does not flip at either width. The peak stays roughly a factor of two below $V_{DD}/2$ and Q returns to 2.9 mV by 4 ns. Sweeping W_A (Fig. 2) puts the upset boundary at $W_A \approx 0.51 \mu\text{m}$, a cell ratio of 0.39 against 1.25 for the sizing as given, so there is about $3.2\times$ margin in access width for this device card, bitline load and timing window.

The failure is abrupt rather than gradual, and the reason is that the access transistor acts as a source follower during a read. Its gate is held at V_{DD} while its source is Q , so as Q rises the overdrive $V_{DD} - V_Q - V_{th}$ collapses and the device throttles itself. The pull-down keeps a full V_{DD} on its gate throughout. That feedback holds Q down over a wide range of widths and then stops working once Q can reach the inverter trip point, which gives a cliff instead of a slope.

Upset is detected from BLB, not from $\max v(Q)$. Once BL has discharged it pulls Q back down, so the peak approaches $V_{DD}/2$ without crossing it and the obvious criterion never fires. BLB can only lose charge if $\overline{Q}$ was pulled down, which requires the cell to have flipped. BLB droop and the collapse of ΔV from 949 mV to 272 mV both appear between $W_A = 0.50$ and $0.52 \mu\text{m}$.

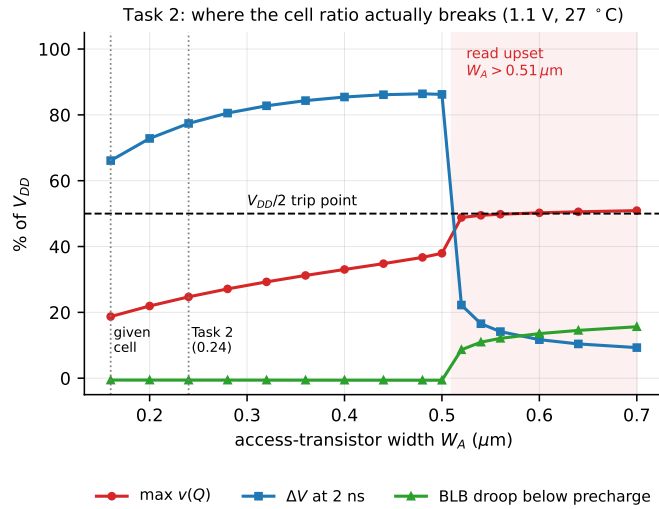


Figure 2: Access-width sweep at 1.1 V, 27 °C.

1.4 Task 3: supply sweep

V_{DD} is stepped from 1.1 V down to 0.6 V in 50 mV steps inside a `.control` loop, at both temperatures. ΔV falls almost linearly with V_{DD} . That linearity is itself the result: the read is discharge limited rather than current limited over this range, because BL is pulled nearly to ground at every supply, so ΔV follows the precharge level V_{DD} instead of the strongly non-linear $I_{on}(V_{DD})$.

At 180 fF, ΔV never reaches the 25 mV offset anywhere in the requested range. The minimum is 106 mV at 0.6 V and 85 °C, over $4\times$ the offset. Extrapolating the four lowest points linearly puts a crossing near 0.50 V at 27 °C and 0.51 V at 85 °C, but those are extrapolations rather than simulated points, and both sit near where V_{DD} approaches $V_{th} + |V_{th,p}|$ and the linear trend has to break down. Read disturb moves the other way. $\max v(Q)$ falls from 19 % of V_{DD} at 1.1 V to 12 % at 0.6 V, because the access and pull-down devices weaken together while the window stays fixed, and because the source-follower shutoff above bites harder when V_{th} is a larger fraction of the rail. The two

Table 4: Supply sweep at 27 and 85 °C, $C_{BL} = 180$ fF.

V_{DD} (V)	$\Delta V(\text{BLB, BL})$ at 2 ns (mV)			$\max v(Q)$ (% of V_{DD})	
	27 °C	85 °C	Δ	27 °C	85 °C
1.10	727	579	-148	19 %	20 %
1.05	671	531	-139	18 %	20 %
1.00	613	483	-130	18 %	19 %
0.95	554	434	-120	17 %	19 %
0.90	495	386	-109	17 %	18 %
0.85	435	337	-98	16 %	17 %
0.80	375	288	-87	15 %	17 %
0.75	315	240	-75	15 %	16 %
0.70	256	192	-63	14 %	15 %
0.65	198	147	-51	13 %	14 %
0.60	142	106	-36	12 %	13 %

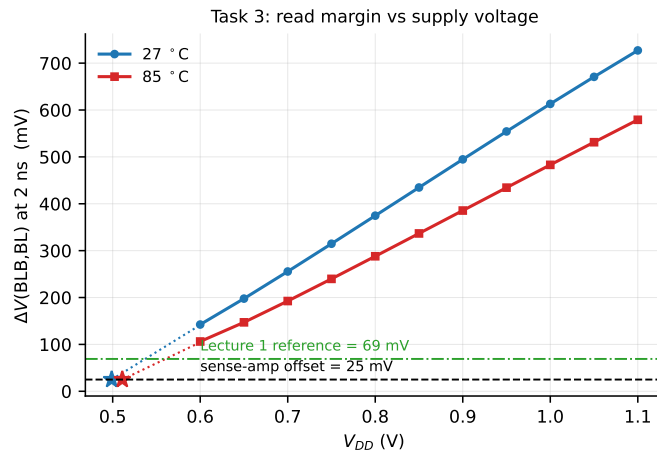


Figure 3: Read margin against supply. Dotted lines extrapolate the four lowest points.

failure mechanisms therefore run in opposite directions with supply. At 1.1 V the cell is easy to read and easier to disturb; at 0.6 V it is stable but the signal is small. There is no single worst corner and both have to be checked.

1.5 Task 4: 85 °C

Raising the temperature to 85 °C at 1.1 V drops ΔV from 727 mV to 579 mV, a loss of 148 mV or -20 %. At the 0.7 V, 85 °C corner $\Delta V = 192$ mV with the storage node disturbed to 15 % of V_{DD} .

ΔV moves far more than the offset does. A latch-type sense amplifier’s input-referred offset is dominated by threshold mismatch in its input pair, with $\sigma V_{th} \propto 1/\sqrt{WL}$ by Pelgrom’s law, and that is fixed by dopant statistics at manufacture rather than by temperature. The 25 mV figure is effectively constant across this range, so the margin closes from one side only: ΔV falls -20 % while the bar it has to clear stays put, and the ratio drops from about $29\times$ to $23\times$. The mechanism on the ΔV side is mobility degradation, roughly $\mu \propto T^{-1.5}$, weakening the access transistor; this outweighs the opposing fall in V_{th} with temperature at these overdrives. The same mobility loss pushes Q up by about 10 %, so read stability degrades at the same time.

1.6 Answer to the central question

At $C_{BL} = 180$ fF the cell reads correctly at 0.7 V and 85 °C with $7.7\times$ margin. On a column loaded as Part B’s own geometry implies, 345 fF, it still reads correctly but the margin is $4.1\times$. Read stability is the tighter of the two constraints either way, since Task 2 left only $3.2\times$ in access width against more than $4\times$ in sense margin.

Two limits are worth stating. Every number here is one simulation at nominal process, so the σV_{th} spread across the 2^{24} cells of a 2 MB array, which is what actually decides whether the array works, is not represented at all. And the answer depends on the device card: the two PTM cards differ by $2.9\times$ on the same netlist, and the high-performance card used here is the one that matches the `itrs-hp` setting given to CACTI in Part B.

Carried into Part B: cell read current ≈ 131 μ A, which sets the bitline delay CACTI reports and is used in the Task B4 hand check.

2 Part B: CACTI, sizing the 2 MB SRAM L2

2.1 Setup

CACTI 7 with the configuration given in the assignment: 2 MB capacity, 64 B blocks, 8-way associativity, one read/write port, 4 UCA banks, 45 nm, 350 K, `itrs-hp` for both the data-array cells and the periphery, cache type `cache`, and ED² as the optimisation target with design objective weights 0:0:0:100:0. Everything else is left at CACTI’s defaults.

CACTI reports leakage per bank; with 4 banks the whole-cache figure is four times the reported value, and that is what is quoted here so it can be compared with NVSim in Part C.

Table 5: Part B baseline. This is the SRAM reference for Parts C–E.

Metric	2 MB SRAM L2, 45 nm
Access time (ns)	2.90
Cycle time (ns)	2.65
Dynamic read energy (pJ/access)	792.9
Dynamic write energy (pJ/access)	851.2
Leakage, whole cache (mW)	2251
Total area (mm ²)	10.66
Area efficiency (%)	54.3
Winning organisation (Ndlw/Ndbl/Nspd)	4 / 2 / 1
Tag organisation (Ntwl/Ntbl/Ntspd)	2 / 2 / 2

2.2 Task 1: baseline

Access time is 2.90 ns, dynamic read energy 793 pJ, whole-cache leakage 2251 mW, and area 10.66 mm² summed over the data and tag arrays at an efficiency of 54.3 %. CACTI also prints a bounding box of 2.213×5.184 mm, i.e. 11.47 mm² of occupied silicon, and that larger figure is the one used for the iso-area comparison in Parts C and E, since it is what the cache actually costs on die. The winning organisation is $N_{dwl}/N_{dbl}/N_{spd} = 4/2/1$. At a 2 GHz clock the access time is 6 cycles, which is the L2 hit latency used in Part E.

The delay breakdown is more informative than the total. Of the 2.90 ns, 47 % is H-tree wire delay into and out of the bank (1.36 ns combined), 0.54 ns is decoder and wordline, and only 14 % (0.407 ns) is the bitline itself. Sense amplification is negligible. At this capacity the access is dominated by interconnect rather than by the array, which is the single most useful fact carried forward: it is why swapping the bitcell in Part C changes read latency so little.

2.3 Task 2: capacity sweep

Capacity is swept from 256 kB to 16 MB with every other parameter fixed (Table 6, Fig. 4).

Table 6: Capacity sweep, all other parameters fixed.

Capacity	$\log_2(\text{bytes})$	Access (ns)	Read (pJ)	Leak (mW)	Area (mm ²)
256 kB	18	2.42	658	361	4.86
512 kB	19	2.48	679	633	5.69
1 MB	20	2.63	717	1175	7.36
2 MB	21	2.90	793	2251	10.66
4 MB	22	3.53	930	4409	17.29
8 MB	23	4.47	1220	8735	34.30
16 MB	24	6.36	1742	16978	65.44

Amrutur and Horowitz’s “one gate delay per doubling” is not visible here, and not marginally so. The rule predicts a constant increment of about one FO4 delay, roughly 20 ps at 45 nm. What the sweep shows instead is an increment that keeps growing: 67 ps from 256 kB to 512 kB, rising to 1.89 ns from 8 MB to 16 MB. The mean cost of a doubling is 597 ps, about 30 gate delays, and the curve is convex on a $\log_2$ axis where the rule predicts a straight line.

The reason is in the delay breakdown. The rule describes a decoder-depth limited array, where each doubling adds one more level of decode logic and so one gate delay, and that is what dominated at the 0.25 to 0.5 μm nodes where it was measured. At 45 nm a multi-megabyte array is interconnect limited instead. The H-tree is the largest single term at every capacity in the sweep, and wire delay per unit length does not improve with scaling, it gets worse. Doubling the capacity lengthens the array by $\sqrt{2}$ in each dimension and RC delay goes with the square of length, so the wire terms compound while the logic-depth term stays logarithmic. The claim is not wrong, it has been overtaken by which component sits on the critical path.

2.4 Task 3: the objective

The same 2 MB cache is re-run three times with only the objective changed: pure delay (100:0:0:0 with ED optimisation off), pure area (0:0:0:100, likewise), and ED²P.

Table 7: Same cache, three objectives.

Objective	$N_{dwl}/N_{dbl}/N_{spd}$	Access (ns)	Read (pJ)	Area (mm ²)
Pure delay (100:0:0:0)	4 / 2 / 1	2.89	898	10.21
Pure area (0:0:0:100)	2 / 2 / 1	3.36	797	8.65
ED ² P	4 / 2 / 1	2.90	793	10.66

Delay and ED²P both select 4/2/1. Pure area selects $N_{dwl} = 2$ instead, trading $1.2\times$ the access time for 15 % less area (10.21 mm² to 8.65 mm²).

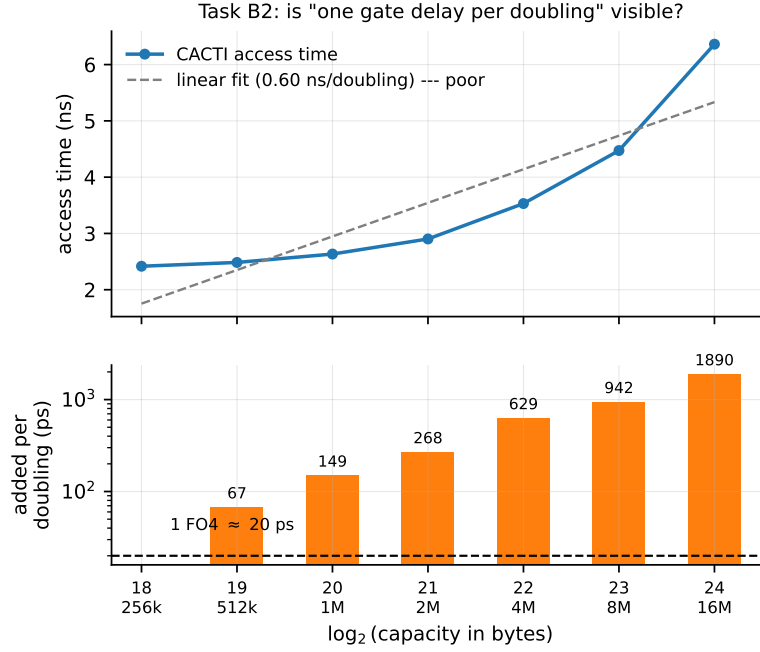


Figure 4: Access time against capacity (top) and delay added per doubling (bottom, log scale). A constant one-gate-delay increment would appear as flat bars.

N_{dwl} is the number of vertical cuts through the array, so it sets how many segments each wordline is divided into. Doubling it halves the wordline length each row driver has to charge and roughly quarters that RC delay. Every cut also replicates the row decoder, the wordline drivers and a sense-amplifier strip along the new boundary, and none of that stores any bits. The delay objective accepts the overhead because it scores only time, and the area objective refuses it because it scores only area.

The general point is that there is no optimal cache, only the optimum of the objective written down. The same 2 MB specification produces a 2.89 ns cache, a 3.36 ns cache, or the ED²P baseline, and CACTI is right in all three cases. Quoting “the access time of a 2 MB L2 at 45 nm” without naming the objective builds in an error bar the size of the spread. The baseline used here is ED²P optimal and 15 % larger than it needs to be if area is what matters, which it is in Part C.

2.5 Task 4: bitline delay hand check

CACTI reports 0.407 ns of bitline delay for the Task 1 winner. The Lecture 4 estimate for a distributed RC line is $t = 0.38rcL^2$. Using the winning subarray’s bitline length $L = 336 \mu\text{m}$, a minimum-pitch local wire with $r = 6.0 \Omega/\mu\text{m}$ (copper, $\rho = 2.2 \times 10^{-8} \Omega\text{m}$, $45 \text{ nm} \times 81 \text{ nm}$ cross-section) and $c = 0.40 \text{ fF}/\mu\text{m}$ including the cell drain junctions, the estimate is 0.104 ns. CACTI’s value is $3.9\times$ larger.

Two effects account for the gap, and both make the hand figure optimistic.

The first is capacitance the formula does not count. $0.38rcL^2$ uses the wire only, but the subarray is $336 \mu\text{m}$ tall at a cell height of $0.657 \mu\text{m}$, so 512 unselected cells hang off each bitline. Their drain junctions at roughly 0.5 fF each add about 256 fF to the 89 fF of wire, so the real load is close to 345 fF, nearly four times the wire alone. This is the same correction Part A’s bitline sweep needed.

The second, and the one that matters more, is that the bitline is not driven by a voltage source at all. $0.38rcL^2$ is the 50 % step response of a distributed RC line fed by an ideal driver that can supply whatever current the line demands. A real read has one access transistor sinking the 131 μA that Part A measured for exactly this device into that load, so the event is current limited and the delay is $C_{BL}\Delta V/I_{cell}$, a ramp rather than an RC settling. CACTI computes the time to develop the differential the sense amplifier actually needs at that current, which no purely RC estimate can reproduce. The lumped formula is the right tool for a wire, and a bitline is a wire plus 512 loads

plus a current source.

3 Part C: NVSim, the same 2 MB as STT-MRAM

3.1 Setup

NVSim at 2 MB, 8-way, 512-bit word, 45 nm, 350 K, HP device roadmap, H-tree routing, sequential cache access mode, read-EDP optimisation target. The array organisation and peripheral search are NVSim’s own; only the bitcell differs from Part B.

The cell file is the one specified in the assignment: MRAM, 54 F^2 cell area, aspect ratio 2.0, $R_{on}/R_{off} = 3/6\text{ k}\Omega$, current read mode at $40\text{ }\mu\text{A}$, $145\text{ }\mu\text{W}$ read power, current set/reset at $200\text{ }\mu\text{A}$ for 10 ns, CMOS access device of width 6 F.

NVSim additionally requires four fields the assignment’s listing omits: `ReadVoltage`, `MinSenseVoltage`, `SetEnergy` and `ResetEnergy`. These are taken from NVSim’s shipped `sample_STTRAM.cell`, with `MinSenseVoltage` set to 25 mV so the sense-amp offset matches the one used in Part A.

Table 8: 2 MB L2 at 45 nm: SRAM (CACTI) against STT-MRAM (NVSim). SRAM leakage is the whole-cache figure.

Metric	SRAM (CACTI)	STT-MRAM (NVSim)	Ratio
Read latency	2.90	2.53	$0.87\times$
Write latency	2.90	10.53	$3.63\times$
Read energy	792.88	562.00	$0.71\times$
Write energy	851.22	1133.00	$1.33\times$
Leakage	2250.53	433.93	$0.19\times$
Area	10.66	2.89	$0.27\times$

3.2 Tasks 1 and 2: comparison with the SRAM baseline

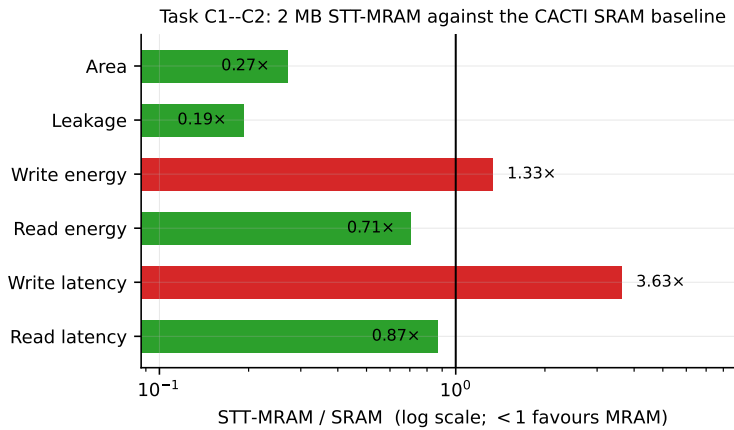


Figure 5: STT-MRAM relative to SRAM per metric, log scale. Values below 1 favour MRAM.

Two metrics improve by more than $3\times$ and two degrade.

Area and leakage improve. Area falls $3.7\times$ (10.66 mm^2 to 2.89 mm^2) and leakage $5.2\times$ (2251 mW to 434 mW). Both follow from the bitcell. A 6T SRAM cell is six transistors, roughly 140 F^2 , and retains its value through a cross-coupled pair permanently connected to the rail, which draws current continuously. An MTJ with one access transistor is 54 F^2 and retains its value in the magnetisation of the free layer, which draws none. The remaining 434 mW is peripheral, decoders, sense amplifiers, H-tree drivers, so the leakage ratio would improve further at larger capacities.

Write latency and write energy degrade. Writes take $3.6\times$ longer (2.90 ns to 10.53 ns) and cost $1.3\times$ more energy. This is switching physics, not periphery. Writing an SRAM cell means overpowering a latch, which takes about a gate delay. Writing an MTJ means passing enough current for long enough to reverse the free layer by spin-transfer torque, a thermally activated process whose

switching probability depends on the overdrive above the critical current and on the pulse duration. The cell file specifies 200 μA for 10 ns, and that 10 ns pulse accounts for nearly all of the 10.53 ns write latency.

Read latency and read energy barely move ($0.87\times$ each, marginally in MRAM’s favour). This follows from Part B: 47 % of the SRAM access is H-tree wire delay, which is unaffected by the bitcell, so changing the cell changes only the array-internal fraction. The comparison does cross two tools with different peripheral assumptions, NVSim is in sequential access mode here, so the supportable statement is that read latency is comparable, not that MRAM is 7 % faster.

3.3 Task 3: TMR ratio

Re-running with $R_{on} = 4\text{ k}\Omega$ and $R_{off} = 12\text{ k}\Omega$ (TMR 3:1 instead of 2:1) changes read latency from 2.53 ns to 2.54 ns (+0.3 %), read energy from 562 pJ to 561 pJ, and area from 2.89 mm^2 to 2.88 mm^2 . Sweeping the ratio from 1.5:1 to 6:1 moves read latency by 0.5 % in total and leaves the other outputs unchanged. Read latency is the metric that moves most, by less than the tool’s error bar.

The reason is a threshold, and both designs are already well past it. With current-mode sensing the available signal is $\Delta V_{\text{sense}} = I_{\text{read}}(R_{\text{off}} - R_{\text{on}})$:

- TMR 2:1, $40\text{ }\mu\text{A} \times 3\text{ k}\Omega = 120\text{ mV}$
- TMR 3:1, $40\text{ }\mu\text{A} \times 8\text{ k}\Omega = 320\text{ mV}$

MinSenseVoltage is set to 25 mV. Both configurations clear it by a wide margin, so the sense amplifier resolves in the same time either way and the read path stays dominated by decode, H-tree and sense-amp periphery, none of which the cell resistance touches. Raising TMR only helps while TMR is the binding constraint, which happens at small read current, high temperature, aggressive scaling, or with a noisier amplifier, where it turns an unreadable array into a readable one. Above the threshold the extra margin has already been paid for and cannot be spent on anything else.

What TMR provides physically is sense margin, that is, the separation between the “0” and “1” distributions relative to their spread from MTJ process variation and reference-cell offset. It buys read reliability, so bit error rate and therefore yield and ECC overhead, rather than read speed or read energy. A deterministic array model carries no variation distribution to integrate over, so it cannot price the thing TMR improves. A TMR headline needs a statistical model before it becomes an architectural number.

3.4 Task 4: write current and cell area

Halving **ResetCurrent** from 200 to 100 μA changes area by 0 %: 2.89 mm^2 to 2.89 mm^2 . Sweeping the write current over a wider range shows why (Fig. 6).

Table 9: Cell variants. The last row resizes the access transistor and the cell to match the halved write current.

Cell variant	Read (ns)	Write (ns)	Read (pJ)	Write (pJ)	Leak (mW)	Area (mm^2)
Baseline (3 / 6 k Ω , 200 μA)	2.53	10.53	562	1133	434	2.888
TMR 3:1 (4 / 12 k Ω)	2.54	10.54	561	1132	434	2.884
Write current halved (100 μA)	2.53	10.51	560	1131	434	2.888
100 μA with cell resized to 38.4 F ²	3.66	11.68	299	945	131	1.727

Area is flat at 2.89 mm^2 from 50 to 300 μA , steps up at 400 μA , and reaches 3.31 mm^2 (+15 %) at 800 μA . Below the knee the change is confined to the write drivers and the bitline write energy; the cell itself does not move, because **CellArea** is an input to NVSim rather than something it derives, and the tool never checks that the access transistor declared can actually supply the current declared. The coupling the task asks about therefore has to be closed by hand:

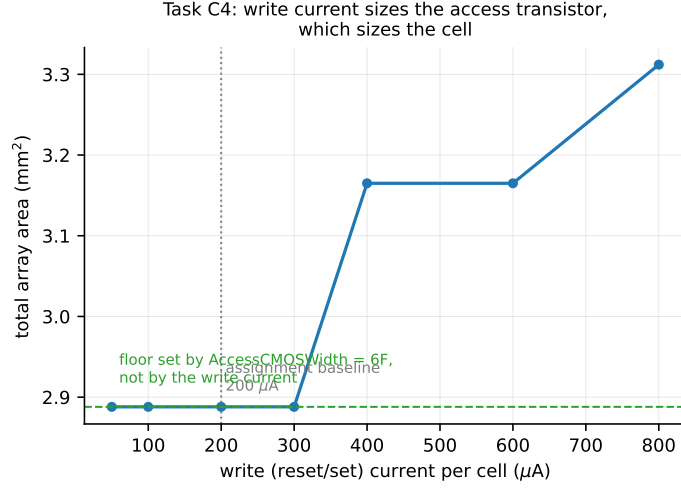


Figure 6: Array area against per-cell write current.

write current $\rightarrow$ access-transistor width $\rightarrow$ cell area $\rightarrow$ array density

The access transistor has to supply the full write current and an MOS device delivers current in proportion to its width, so $W \propto I_{write}$. In a 1T-1MTJ cell the junction sits above the metal stack and takes no silicon of its own, so the cell footprint is the access-transistor footprint.

Working it through for this cell: $54F^2$ at aspect ratio 2.0 is $10.392F \times 5.196F$, and the declared 6F device lies along the long dimension, which leaves $10.392 - 6 = 4.392F$ of contact and spacing. Halving the write current halves the required gate width, so 6F becomes 3F, the long dimension becomes $3 + 4.392 = 7.392F$ and the cell falls to $7.392 \times 5.196 = 38.4F^2$, a reduction of 28.9%. The set current has to halve as well, since the same transistor drives both directions. Feeding that cell back into NVSim gives the last row of Table 9: array area falls from 2.89mm^2 to 1.73mm^2 , leakage from 434mW to 131mW and write energy from 1133pJ to 945pJ. The array shrinks by more than the 28.9% the cell did, because NVSim re-optimises the bank and mat organisation around the smaller cell, and the leakage fall is larger still since subthreshold current through the access device scales with its width.

So the full chain is: write current sets the access-transistor width, the width sets the cell area, the cell area sets array density, and density sets the capacity available at fixed silicon area, which is what Part E shows actually moves program runtime. A device paper that halves switching current is not mainly offering lower write energy, which is the small prize. It is offering a 28.9% smaller cell, and at iso-area that is about $1.4\times$ more capacity.

This is also why the comparison with TMR is worth making. The $3.7\times$ density advantage is what lets 7.4MB of STT-MRAM fit in the SRAM's footprint, and it is a property of the write current, not of the MTJ resistance ratio. A device needing 800µA to switch reliably gives back 15% of the density it was chosen for. Reducing switching current moves the metric the technology is selected for; raising TMR moves a metric this class of tool cannot even resolve.

4 Part D: Ramulator 2.0, the L2 miss stream on DDR4

4.1 Setup

DDR4-3200AA, 8Gb x8 organisation, one channel, two ranks, generic controller with an open-row policy, FR-FCFS scheduling, all-bank refresh and RoBaRaCoCh address mapping. Frontend clock ratio 8, memory clock ratio 3. The relevant timings are $t_{RCD} = 22$, $t_{RP} = 22$, $t_{RAS} = 52$, $t_{CL} = 22$ and $t_{CCD} = 4$ (short) / 8 (long) cycles, with $t_{CK} = 0.625\text{ns}$.

The trace is 57313 reads and 5708 writes recorded on the memory-side port of the Part E baseline L2 (2MB, 8-way), running bfs over a 65k-node Kronecker graph. The graph is built beforehand

and loaded from file, so the recorded region is the traversal itself; generating the graph inside the simulator would fill the trace with cold-miss streaming from a compute-bound phase whose working set is small enough to sit in L1. Evictions that carry no data to DRAM (**CleanEvict**) are excluded; writebacks are kept.

Average read latency is computed as total accumulated read latency divided by the number of read requests the memory system reports. This is a saturated open-loop replay, the frontend injects a request in every cycle it can, with no dependent instruction to stall it, so absolute latencies are dominated by queueing rather than by DRAM timing, and only ratios between configurations are meaningful.

Table 10: Four configurations on the same trace. Latencies in memory cycles.

Configuration	Memory cycles	Avg read (cycles)	(ns)	Row-buffer hit rate	ACT commands
FR-FCFS, RoBaRaCoCh, 1 ch (baseline)	386761	867	542	90.5 %	5954
FCFS, RoBaRaCoCh, 1 ch	732818	1516	947	90.1 %	6195
FR-FCFS, ChRaBaRoCo, 1 ch	881951	1853	1158	91.3 %	5463
FR-FCFS, RoBaRaCoCh, 2 ch	170205	655	410	92.0 %	2515

Part D: scheduler, address mapping and channel count

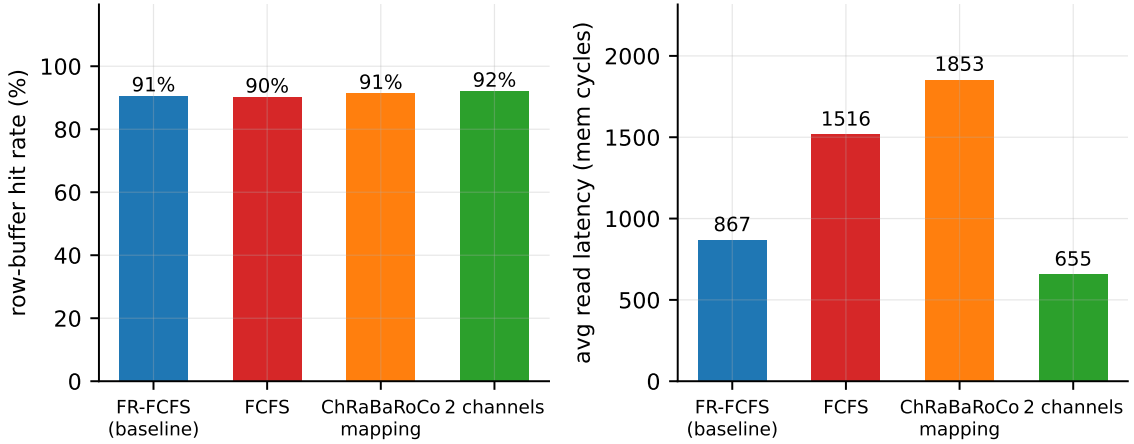


Figure 7: Row-buffer hit rate and average read latency across the four configurations.

4.2 Task 1: baseline

FR-FCFS on one channel completes the trace in 386761 memory cycles with an average read latency of 867 cycles (542 ns) and a row-buffer hit rate of 90.5 %.

4.3 Task 2: FCFS

Switching to strict first-come-first-served costs $1.89\times$ in total cycles (386761 to 732818) and $1.75\times$ in read latency (867 to 1516 cycles, +649 cycles per read).

The row-buffer hit rate does not collapse: it moves from 90.5 % to 90.1 %, and the activate count changes only from 5954 to 6195. So the usual explanation, that FCFS loses row hits and pays $t_{RP} + t_{RCD} = 44$ cycles for each one, does not apply here, since that would require thousands of additional ACT commands and only 4 % more appear.

What is lost is bank-level parallelism. A BFS miss stream over a CSR graph has high spatial locality, because consecutive misses walk an edge list, so most accesses find an open row regardless of the order they are serviced in. The row hits come from the access pattern, not from the scheduler. What FR-FCFS provides is the freedom to let a request to an idle bank overtake one whose bank is still occupied for t_{RAS}/t_{RC} . FCFS forbids that, so the oldest request blocks everything behind it including requests to banks that are free. The +649 cycles per read is queueing behind busy banks.

4.4 Task 3: address mapping

Changing the mapping from **RoBaRaCoCh** to **ChRaBaRoCo** places the row bits below the bank bits in significance, so a sequential address sweep walks every column of a row and every row of a bank before reaching the next bank. This costs $2.28\times$ in total cycles, with a read latency of 1853 cycles, the worst of the four configurations, and worse than FCFS.

Row-buffer behaviour again does not explain it. The hit rate is higher than the baseline (91.3 % against 90.5 %) and there are fewer activates (5463 against 5954).

To quantify what is actually lost, the trace was replayed through both mappers and the bank index extracted for every access. This organisation has $2 \times 4 \times 4 = 32$ independent banks.

Table 11: Bank-level parallelism measured over the real trace.

Address mapping	banks reached (of 32)	mean banks per 64-access window	busiest bank's share of traffic
RoBaRaCoCh	32 / 32	3.75	6.4 %
ChRaBaRoCo	1 / 32	1.00	100.0 %

Bank parallelism is not reduced, it is removed entirely. **RoBaRaCoCh** reaches all 32 banks and touches 3.75 distinct banks in a typical 64-access window. **ChRaBaRoCo** reaches 1 bank, and 100 % of the traffic lands in it. With 10 column bits and 16 row bits below the bank field, the bank index only changes every 2^{26} transactions, which is far beyond the working set, so the whole trace falls into a single bank of 32.

That is why the hit rate rose while throughput collapsed. Serialising everything into one bank makes the queue back up much further, which gives FR-FCFS a larger window in which to batch same-row requests, so the hit rate improves. Meanwhile a single bank cannot overlap one row's t_{RP} and t_{RCD} with another bank's column transfers, and 31 idle banks cannot help however well the one in use is scheduled. The configuration with the best row-buffer hit rate is the slowest in the table, so hit rate on its own is not a useful proxy for DRAM performance.

4.5 Task 4: channel count

Doubling to two channels improves read latency by 24 % (867 to 655 cycles) and more than doubles throughput ($2.27\times$). Extending the sweep gives $4.65\times$ at four channels and $8.36\times$ at eight, with latency levelling off at 536 and 535 cycles.

Table 12: Achieved column throughput against the t_{CCD} floors.

Configuration	accesses per cycle per channel	vs $1/t_{CCD_L}$ (0.125)	vs $1/t_{CCD_S}$ (0.250)
Baseline, FR-FCFS	0.1629	130.4 %	65.2 %
FCFS	0.0860	68.8 %	34.4 %
ChRaBaRoCo	0.0715	57.2 %	28.6 %
Two channels	0.1851	148.1 %	74.1 %

Latency therefore does not barely improve on this trace, and the binding parameter can be identified from the output. It is not t_{RCD} or t_{RP} : with a 90.5 % row-hit rate only about one access in ten activates at all, and the ACT count (5954 against 57313 reads) shows this directly, if activation were the limit, adding channels would not help, because each channel would still pay t_{RCD} per access. The same argument rules out t_{RAS} . The limit at one channel is t_{CCD} : with nine accesses in ten being column reads to an already-open row, the constraint is how fast column commands can be issued onto a single shared data bus, $t_{CCD_S} = 4$ cycles across bank groups and $t_{CCD_L} = 8$ within one. Each additional channel brings its own bus and its own t_{CCD} budget, which is why throughput scales nearly linearly to four channels.

As a general diagnostic: compare the ACT count with the access count to test t_{RCD} and t_{RP} , and if activations are rare, compare the achieved reads-per-cycle against the t_{CCD} -implied bus ceiling. The

flattening between four and eight channels is not a DRAM limit, the replay cannot inject quickly enough to keep eight buses busy.

The miss traffic does not fit comfortably in one DDR4 channel. The superlinear $2.27\times$ from the second channel only occurs if the first was saturated.

5 Part E: gem5, the effect on a running program

5.1 Setup

Single X86 core at 2 GHz in syscall-emulation mode, 32 kB 8-way L1I and L1D at 2-cycle latency, 8-way L2, DDR4-2400 8x8 main memory, 4 GB address space. Two GAPBS kernels, **bfs** and **sssp**, over a 65k-node Kronecker graph (7.8 MB CSR) built beforehand and loaded from file, each run to completion. Loading a prepared graph keeps the simulated region in the traversal rather than in the graph generator, whose working set is small enough that the L2 is never exercised.

L2 capacities and hit latencies come from Parts B and C rather than from assumed values:

- **SRAM, 2 MB.** CACTI access time 2.90 ns, so 6 cycles at 2 GHz, occupying 11.47 mm^2 .
- **STT-MRAM, 8 MB.** NVSim hit latency 3.21 ns, so 7 cycles, occupying 10.32 mm^2 .

8 MB is the iso-area capacity: NVSim fits it in 10.32 mm^2 against CACTI's 11.47 mm^2 for 2 MB of SRAM, so the swap really is $4\times$ the capacity in the same silicon. The measured read penalty is one cycle, not the factor of two that a slower bitcell might suggest, because 47 % of the access is H-tree wire delay and is unaffected by the cell. Since the whole comparison turns on that single number, a third configuration re-runs the MRAM at 14 cycles as a sensitivity check.

Table 13: Out-of-order core. *simTime* is simulated wall-clock.

Kernel	L2 configuration	IPC	L2 miss rate	simTime (ms)
bfs	SRAM 2 MB, 6 cyc	0.648	56.0 %	4.62
bfs	MRAM 8 MB, 7 cyc	0.701	44.0 %	4.27
bfs	MRAM 8 MB, 14 cyc	0.661	43.9 %	4.53
sssp	SRAM 2 MB, 6 cyc	0.749	15.3 %	21.96
sssp	MRAM 8 MB, 7 cyc	0.799	13.1 %	20.58
sssp	MRAM 8 MB, 14 cyc	0.728	13.1 %	22.59

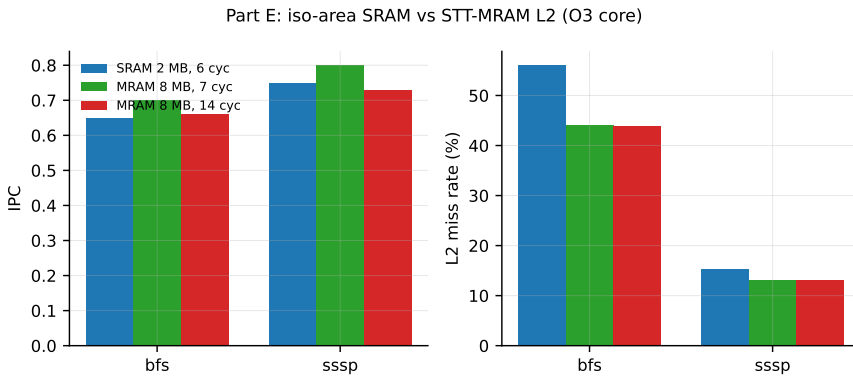


Figure 8: IPC and L2 miss rate for the three L2 configurations.

5.2 Tasks 1 and 2: which kernel benefits

At the measured latencies the MRAM L2 is faster on both kernels: $1.081\times$ on **bfs** and $1.067\times$ on **sssp**. At the pessimistic 14-cycle latency **bfs** still gains ($1.019\times$) while **sssp** loses ($0.972\times$).

The deciding property is the baseline miss rate. **bfs** misses 56.0 % of L2 accesses at 2 MB; its frontier and CSR working set do not fit, so $4\times$ the capacity converts into fewer DRAM accesses (56.0%

to 44.0 %, a 22 % reduction in absolute misses). **sssp** misses only 15.3 % at 2 MB, because its hot working set, the delta-stepping buckets and the distance array, largely fits, so extra capacity removes far less (14 %) while the added hit latency is charged on the $\sim 85\%$ of accesses that already hit.

Capacity pays off on misses and latency is a tax on hits, so the ratio of hits to misses determines the outcome. A kernel that is capacity-bound at the baseline size gains; a kernel that already fits pays the tax with little left to recover.

5.3 Task 3: in-order core

Re-running with the in-order timing CPU reduces the advantage on both kernels: **bfs** from $1.081\times$ to $1.035\times$, and **sssp** from $1.067\times$ to $1.016\times$.

Per-miss accounting explains this. On **bfs**, each avoided L2 miss is worth 99 cycles to the in-order core but only 47 cycles to the out-of-order core. The in-order machine has one miss outstanding at a time, so an avoided miss returns a nearly fully exposed DRAM round trip. The out-of-order core keeps several misses in flight through its MSHRs, so misses overlap and any individual one sits only about half on the critical path. Out-of-order execution was already hiding roughly half the cost of every L2 miss, which is also why the extra hit latency of the MRAM costs so little there.

The relative advantage is nevertheless larger on the out-of-order core, because its baseline is much shorter: **bfs** finishes in 9.24 M cycles against 25.7 M in order, so the same absolute saving is a larger fraction of the run. Memory-level parallelism compresses the absolute benefit of a larger cache and amplifies its relative effect.

5.4 Task 4: the argument

(i) SRAM or STT-MRAM? On this evidence, STT-MRAM as an 8 MB iso-area replacement, though the margin is small and the case rests on capacity rather than on the device being intrinsically better. The chain is as follows. Part A measured 131 μA of read current per 6T cell and confirmed the cell reads correctly at 0.7 V and 85 °C with $7.7\times$ margin to the sense-amp offset, so the SRAM baseline is grounded in a device simulation rather than assumed. Part B sized that baseline at 2.90 ns, 793 pJ per read, 2251 mW of leakage and 10.66 mm², and showed that 47 % of the access is wire rather than array. Part C placed 8 MB of STT-MRAM in the same area with $5.2\times$ less leakage and $3.7\times$ the density, at $3.6\times$ the write latency. Because the access is wire-dominated, the read penalty for that swap is one cycle. Part D showed the resulting miss stream needs more than one DDR4 channel, with two channels giving $2.27\times$. Part E converts this into $1.081\times$ on **bfs** and $1.067\times$ on **sssp**, plus a standing leakage saving that no IPC figure captures.

Stated precisely: 8 MB of STT-MRAM is 6–8 % faster than 2 MB of SRAM on **bfs** and **sssp**, in the same area, given these device numbers, this cell file, an FR-FCFS scheduler and an out-of-order core, and it is 3 % slower on **sssp** if the hit-latency penalty is 7 cycles rather than the 1 measured here. It is a capacity argument with a leakage benefit, and it is sensitive to write-heavy workloads that were not run.

(ii) Three assumptions to attack first. *A deterministic corner is not a yield.* Every Part A number is one simulation at one process point. A 2 MB array holds 2^{24} cells, and what decides whether the cell works is the σV_{th} tail across all of them, roughly 6σ of per-cell margin. The $7.7\times$ margin at the worst corner and $3.2\times$ in access width are both single-point results, and Part C’s TMR sweep moved nothing precisely because NVSim carries no variation model with which to price sense margin. The reliability case is unmodelled throughout.

The cell file. The density advantage rests on **CellArea** of 54 F² and **AccessCMOSWidth** of 6 F, both asserted rather than derived. Task C4 showed area is flat in write current below 400 μA and then steps up, so a device needing 800 μA to switch reliably returns 15 % of the density the technology was chosen for. The $4\times$ capacity on which the Part E result depends is one line of a text file.

Workload coverage. Both kernels run a 65 k-node graph to completion on one core, and STT-MRAM’s weakest property by a wide margin is writing ($3.6\times$ latency, $1.3\times$ energy), which a read-dominated graph traversal barely exercises. A write-heavy or multi-core workload could remove the 6–8 % advantage entirely. The cross-tool comparison deserves the same scrutiny: CACTI’s access time and NVSim’s hit latency rest on different peripheral assumptions, so the one-cycle penalty that makes this argument work is a difference between two models rather than a measurement of two fabricated caches.